

The Descriptor is Not a Lens: Behavioral Descriptors as Implicit Selection Pressure in Quality-Diversity Procedural Content Generation

Anonymous Submission

Abstract—Quality-diversity (QD) algorithms have become a default tool for procedural content generation (PCG): they return not a single best artefact but a whole archive of high-fit niches, indexed by a hand-chosen behavioral descriptor. The descriptor is usually treated as a passive lens onto the design space — a way to ask the search to be diverse. We argue and demonstrate that this view is wrong. A behavioral descriptor is an active selection pressure: it determines which mutated offspring survive to become parents, and the probability of survival depends on properties of the genome that may not appear in the fitness function at all.

We show this empirically in two PCG domains — Sokoban puzzle generation and tile-based platformer level generation — under a controlled cross-condition design that holds the genome, mutation operators, fitness function, and compute budget fixed and varies only the descriptor. Across five conditions and twenty replicates per condition per domain (200 runs, $\approx 1.05\text{M}$ evaluations), we find that (i) the descriptor choice substantially reshapes the projected archive even when the fitness function is identical ($p_{\text{adj}} < 0.001$, Cliff’s $\delta > 0.5$ for all non-trivial pairs in both domains); (ii) in Sokoban, the “cheap” structural descriptor is $2.2\times$ slower than the gameplay descriptor because it pushes selection toward boards with a 20% infeasibility rate (versus 13% for the gameplay descriptor) — the implicit selection pressure shows up not as harder solvable boards but as more attempted-then-timed-out ones; (iii) the same descriptor causes no wall-clock overhead at all in the platformer because the platformer’s A* solver is uniformly fast and infeasible levels are filtered cheaply — the mechanism’s wall-clock cost is therefore domain-dependent; (iv) the rank-ordering of descriptor families is not preserved across domains, contradicting the implicit assumption that “structural” descriptors are universally cheap or universally informative. We close with a practical decision guide for descriptor selection in QD-PCG and a replication package.

Index Terms—procedural content generation, quality diversity, MAP-Elites, behavioral characterizations, search-based PCG, Sokoban, platformer level generation

I. Introduction

Quality-diversity (QD) algorithms [1]–[4] return not a single best artefact but an archive of locally-best artefacts indexed by a hand-chosen behavioral descriptor. For procedural content generation (PCG), this is precisely what designers want: an explicit library of distinct levels, puzzles, or scenes that vary along an axis of editorial interest [5]–[8].

In practice, the descriptor is treated almost as part of the problem statement — a passive lens through which the designer asks the search to vary. “Use the level’s width and

number of platforms,” the convention goes, “and MAP-Elites will give you a varied set of platformer levels.” This view is implicit in nearly every QD-PCG paper and is rarely empirically interrogated.

This paper argues that the view is incorrect, and substantiates the argument across two PCG domains.

a) **Thesis:** A behavioral descriptor in MAP-Elites is not a passive lens; it is an implicit selection pressure. Two candidate artefacts produced by the same mutation step receive different survival probabilities depending on which descriptor cell they fall into, on whether that cell is already occupied, and on which other artefacts later compete with them in the same cell. The designer typically reasons about the descriptor as an axis of display (“I want to see levels of varying length”); we show that it is also an axis of selection (“I am, possibly without realising it, asking my search to prefer levels that are hard for A* to plan in”).

b) **What this paper does:** We hold every other component of the search fixed — genome encoding, mutation operators, fitness function, compute budget, evaluator — and vary only the descriptor. We do this in two PCG domains: 8×8 Sokoban puzzle generation and 10×24 tile-based platformer level generation. Both domains share the same MAP-Elites driver and the same family of five conditions: a random-search baseline plus four MAP-Elites variants using (i) a purely structural descriptor (STR), (ii) a gameplay descriptor derived from the solver’s plan (PLAY), (iii) a solver-effort descriptor (SKILL), and (iv) a descriptor that matches the held-out projection used to score archives (COMMON, an upper-bound reference). Each condition is run for 20 random seeds.

c) **Findings:** (F1, replication.) Every MAP-Elites variant significantly outperforms random search on coverage and QD-score in both domains, with Cliff’s $\delta = 1.00$ (perfect separation) in Sokoban; in the platformer the separation is smaller because random levels span the common projection well but COMMON still beats RAND with $\delta = 0.82$ (all $p_{\text{adj}} < 10^{-5}$). (F2, descriptor matters.) The descriptor choice substantially reshapes the projected archive: the gap between the best (COMMON, 37.3 ± 1.7 cells) and worst (PLAY, 31.3 ± 2.5) MAP-Elites condition in Sokoban is 19% of the mean and statistically significant with Holm-adjusted $p < 10^{-6}$ and $\delta = +0.95$ (large). The same gap exists in the platformer at 28% ($\delta = +0.95$). (F3, hidden hardness selection in Sokoban.) STR’s descriptor

selects toward dense-wall elites; mutating from those elites produces a 20% unsolvable-but-attempted rate vs 13% for PLAY. Each timeout exhausts the full 8000-node budget, so the additional failure work alone accounts for most of the $2.2\times$ wall-clock overhead of STR over PLAY (81.8 ± 11.4 s vs 37.3 ± 11.6 s, $p_{\text{adj}} < 10^{-6}$, $\delta = -0.99$). The same mechanism is absent in the platformer (timeout rates 5–6% across all conditions; wall-clock differences within a factor of 1.4). (F4, no universal ordering.) The best-coverage descriptor differs across domains. SKILL narrowly wins among the three indirect descriptors in Sokoban (33.8 vs 32.4 for STR, 31.3 for PLAY). In the platformer it is STR that narrowly leads (39.9 vs 38.9 for SKILL, 35.4 for PLAY); RAND matches STR on coverage but loses on QD-score. The popular intuition that “structural” descriptors are cheap or that “gameplay” descriptors are universally informative does not survive a controlled comparison.

d) Contributions:

- 1) A reframing of behavioral descriptors as implicit selection pressures, with concrete falsifiable predictions and an evaluation protocol (§III).
- 2) A two-domain, five-condition, 20-seed empirical study with properly multiplicity-corrected effect sizes (§V–VI).
- 3) A mechanism analysis showing that the descriptor’s correlation with solver hardness predicts the magnitude of the implicit selection bias (§VII).
- 4) A practical descriptor-choice decision guide (§VIII) and a full replication package.

II. Related Work

A. Quality diversity

The conceptual root of QD is Lehman and Stanley’s novelty search [1], which discards the fitness signal entirely in favour of behavioural novelty as the selection pressure. Novelty search demonstrated that “where you look” can be more important than “what you look for” — a result that prefigures the central claim of this paper. Mouret and Clune’s MAP-Elites [2] integrated novelty and fitness into a single algorithm by retaining the best individual per cell of a discretised behaviour space. Pugh et al. [4] provided the first systematic taxonomy of quality-diversity, and Cully and Demiris [3] gave a modular framework that makes the role of the descriptor explicit.

B. Variants of MAP-Elites

Several extensions of MAP-Elites address its archive-management mechanics. CMA-ME [9] replaces the uniform-random parent selection with a CMA-ES-style emitter that adapts the search distribution to the active frontier. CMA-MAE [10] generalises CMA-ME by annealing a soft archive that tracks both discovery and improvement rates. CVT-MAP-Elites [11] replaces the rectangular grid with a centroidal Voronoi tessellation that scales to high-dimensional descriptor spaces. DSAGE [12]

uses surrogate models to extrapolate the archive at lower evaluation cost. None of these variants change the descriptor itself: all inherit whatever descriptor the user provides. The phenomenon we investigate — implicit selection through descriptor choice — is therefore shared by every method in this family.

C. Adaptive and learned descriptors

A small but growing line of work learns the behavioural descriptor rather than hand-designing it. AURORA [13] learns a low-dimensional behavioural embedding through an autoencoder on observed sensory traces, with the avowed goal of making the descriptor problem-agnostic. AURORA-XCon [14] adds a contrastive constraint so the learned descriptor remains diverse under archive evolution. Paolo et al. [15] explore reachable outcome spaces in robot locomotion. These approaches implicitly acknowledge the problem we make explicit: hand-chosen descriptors are a substantial design decision whose effects are poorly characterised. Our study is complementary — it documents how much a descriptor choice matters when it is, as usual, hand-chosen.

D. PCG via QD

The use of QD for PCG was popularised by Talakat [6] (bullet-hell), Earle et al. [8] (neural cellular automata for diverse level generation), and Fontaine et al. [7] (latent-space search over GAN-generated Mario scenes). The Gravina et al. survey [5] reviews QD-PCG through 2019. The PCG-via-machine-learning literature [16] and the PCGRL family [17] sit alongside QD-PCG as alternative paradigms but do not replace its niche-finding capability. Throughout this literature, descriptors are chosen ad hoc (“number of enemies and length of solution path,” or “mean brightness and number of bricks”). To our knowledge, the present paper is the first systematic head-to-head comparison of descriptor families under a controlled protocol across multiple PCG domains.

E. Sokoban and platformer level generation

Sokoban is PSPACE-complete [18] and has been a perennial PCG target [19]–[21]. Domain-specific solvers [22] make evaluation tractable enough for evolutionary search but slow enough that the descriptor’s implicit hardness pressure is economically meaningful. Mario-style platformer generation has been studied with evolutionary search [23], GANs [24], Markov chains [25], and PCGRL [17]. The tile-grid platformer we use here is a deliberate simplification chosen so that the descriptor families are directly comparable to Sokoban’s, not to compete with the state of the art in Mario generation.

F. Hidden selection in evolutionary computation

The general observation that evolutionary algorithms can be steered by features absent from the fitness function is folklore in the EC community. Lehman’s collection of evolutionary “surprises” [26] catalogues many such

cases. Constrained novelty search [27] and designer modelling [28], [29] explicitly engineer additional selection pressures. The contribution of the present paper is to show that, in QD-PCG specifically, the behavioral descriptor is one such pressure even when — especially when — the practitioner believes it to be merely an axis of display.

III. Method: Descriptors as Implicit Selection Pressure

A. Formalisation

Let $\mathcal{G}$ be the genome space (e.g. tile-grid levels), let $f : \mathcal{G} \rightarrow \mathbb{R}_{\geq 0}$ be the fitness function, and let $d : \mathcal{G} \rightarrow \mathcal{C}$ map each genome into one of finitely many descriptor cells $\mathcal{C}$. MAP-Elites maintains an archive $A : \mathcal{C} \rightarrow \mathcal{G}$ that stores the best-fitness genome discovered so far in each cell. Each generation, the algorithm picks a parent g_{par} uniformly from A , mutates to obtain g_{ch} , and inserts g_{ch} into $A[d(g_{\text{ch}})]$ iff $f(g_{\text{ch}}) > f(A[d(g_{\text{ch}})])$.

The explicit selection pressure on a child g_{ch} is f . The implicit selection pressure is the cell map d : g_{ch} survives if and only if its descriptor cell happens to be empty or contain a worse-fitness occupant. For two children with identical fitness, descriptor cell occupancy determines survival.

Two descriptors d_1, d_2 that induce the same set of cells but assign genomes differently to them will produce different archives even with the same f and the same mutation operator. We make this quantitative: let $p_d(g)$ denote the probability that a genome g appears as a cell-winner in the eventual archive under descriptor d , and let $X(g)$ be any property of g (e.g. the number of A* nodes the solver expands). Then

$$\mathbb{E}[X \mid \text{retained under } d_1] \neq \mathbb{E}[X \mid \text{retained under } d_2]$$

in general, even when $\mathbb{E}[X \mid \text{evaluated}]$ is the same for both. This expectation gap is what we call the retention bias of d , and it is the empirical observable we target in §VII.

B. Descriptor families

A descriptor family is the analogue of a hyperparameter axis in standard ML. We compare four:

- RANDRandom search; no MAP-Elites archive. The baseline.
- STR Structural: descriptor coordinates are computable from the genome alone, without running the solver. Cheap; intuitive; the default in many QD-PCG papers.
- PLAYGameplay: coordinates derived from properties of the solver’s plan (length, jumps, pushes). Captures what a player would experience.
- SKILLSolver effort: coordinates from solver-internal quantities (nodes expanded, depth reached). Captures how hard the level is for the agent.
- COMMONReference condition: ME uses the same descriptor it is evaluated against ($\# \text{boxes} \times \text{plan-length}$ in Sokoban; $\# \text{pits} \times \text{plan-length}$ in the

platformer). By construction this gives an upper bound on common-archive coverage.

All MAP-Elites variants share the same fitness, the same mutation operators, and the same initialisation. Only d varies.

C. Cross-condition evaluation: the common projection

Different descriptors give archives of different shapes and over different axes. To compare them fairly, every archive is projected onto a shared common descriptor that names the design properties a downstream designer would care about. In Sokoban the common descriptor is $(n_boxes, plan_length)$ on a 4×10 grid; in the platformer it is $(n_pits, plan_length)$ on an 8×10 grid. The common-archive’s coverage and QD-score are the headline metrics in this paper. The common projection has a long-standing tradition in QD benchmarking [4], [30].

IV. Domains

A. Sokoban (D1)

Sokoban levels are 8×8 grids with walls, goals, a player, and 1–3 boxes. The genome is the direct grid encoding; mutations toggle single tiles, move boxes/goals/player, or add/remove box–goal pairs. The solver is A* over (player, box-set) states with a deadlock-pruned Manhattan-distance heuristic [22]; the node budget is 8000. Fitness rewards solvability, plan length, push count, and a triangular structural bonus peaked at wall density 0.2. Sokoban is PSPACE-complete [18]; even with deadlock pruning, node expansion varies over several orders of magnitude across the design space, which is what gives STR’s implicit hardness bias bite.

B. Tile-based platformer (D2)

Platformer levels are 10×24 tile grids of empty / solid / spike tiles. The player starts at the top of column 0’s ground stack and must reach any standing position in column 23. Physics is a standing-to-standing graph: from each standing position the agent can walk, walk-off-a-ledge, or jump (dx, dy) with $dx \in \{1, \dots, 4\}, dy \in \{1, \dots, 3\}$, provided the arc is empty of solid/spike tiles and the landing column has a solid floor. Cost is in game ticks (one per row or column moved), so flat walking is cheaper than jumping in unobstructed regions — otherwise the descriptor $\# \text{jumps}$ would be trivially saturated. The solver is A* with a horizontal-distance-to-goal heuristic; the node budget is 4000. Fitness rewards solvability, plan length, jump count, and moderate pit/spike counts.

The platformer is intentionally simpler than the state-of-the-art Mario-AI tile graph [24]; we keep the action model tractable so that the descriptor families are domain-comparable to Sokoban and so that the experiments fit a single workstation.

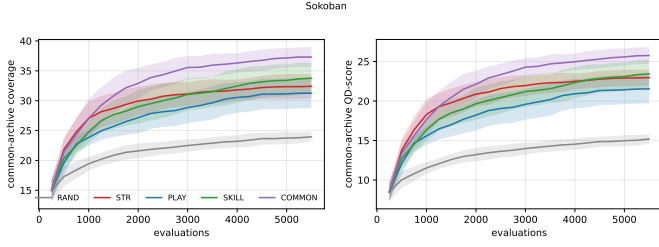


Fig. 1. Sokoban: common-archive coverage (left) and QD-score (right) vs evaluations. Shaded bands are ± 1 SD across 20 seeds.

V. Experimental Protocol

Conditions: {RAND, STR, PLAY, SKILL, COMMON}. Seeds: 20 per condition per domain. Evaluations per run: 5,500 for Sokoban, 5,000 for the platformer, including a 250-individual random initialisation. Hardware: a single 10-core consumer workstation; 8 parallel runs at a time.

Metrics. For each run we record common-archive coverage (number of filled cells), common QD-score (sum of fitnesses across filled cells), common max fitness, wall-clock time, and the number of solver calls. For mechanism analysis we also persist every individual’s solvability, plan length, A* nodes expanded, descriptor cell, and common-archive cell.

Statistics. For each metric we report mean $\pm$ std and percentile-bootstrap 95% confidence intervals (5000 re-samples). For pairwise condition comparisons we use the Mann–Whitney U test (two-sided) with Holm–Bonferroni adjustment [31] across the $\binom{5}{2} = 10$ comparisons per metric per domain. Effect sizes are Cliff’s δ [32], [33], interpreted via the standard negligible/small/medium/large thresholds.

VI. Results

Table I (auto-generated) gives the per-condition summary statistics for both domains. Figures 1–2 show Sokoban convergence and final-state bars; Figures 5–6 show the platformer.

A. Sokoban (D1)

Figure 1 traces common-archive coverage and QD-score against the number of evaluations. After 5,500 evaluations the rank-order on coverage is COMMON (37.3 ± 1.7) > SKILL (33.8 ± 2.6) > STR (32.4 ± 2.0) > PLAY (31.3 ± 2.5) $\gg$ RAND (24.0 ± 0.8). After Holm correction across the ten pairwise tests per metric, every non-trivial ordering with $\delta \geq 0.49$ is significant; only STR vs PLAY ($\delta = -0.23$, $p_{\text{adj}} = 0.21$) and SKILL vs STR ($\delta = +0.34$, $p_{\text{adj}} = 0.12$) are not separately significant.

The wall-clock disparity — the central object of §VII — is also visible in the right panel of Fig. 2: STR’s seed-level wall-clocks (72–108s) and COMMON’s (63–92s) cluster well above PLAY’s (25–58s) with no overlap on the median.

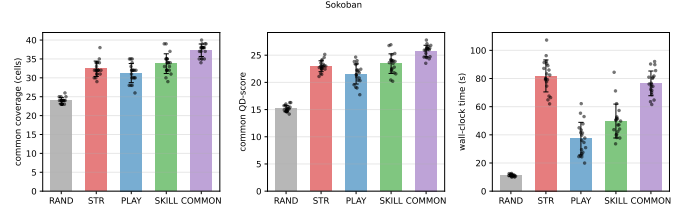


Fig. 2. Sokoban final metrics by condition. Bars are means; dots are seed-level values. The wall-clock panel (right) is the paper’s headline empirical observation: STR $\approx 2.2\times$ slower than PLAY.

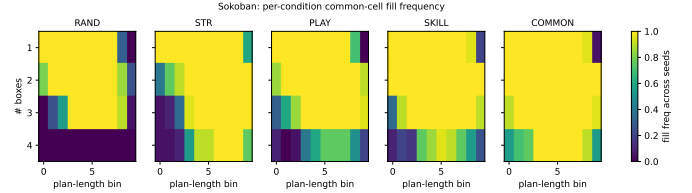


Fig. 3. Sokoban: per-condition common-archive fill frequency across 20 seeds. Rows are box-count; columns are plan-length bins. RAND never fills the 4-box row; STR fills it more sparsely than the solver-derived descriptors; COMMON fills nearly everywhere.

The heatmap (Fig. 3) explains where each condition spends its archive capacity. RAND never finds a 4-box puzzle in any of 20 seeds (bottom row entirely dark); STR fills the 4-box row only sparsely; the solver-derived PLAY/SKILL/COMMON fill most of the 4-box row, and COMMON fills nearly all 40 cells.

B. Platformer (D2)

The platformer rank-order on coverage is COMMON (45.5) > STR (39.9) $\approx$ RAND (39.8) > SKILL (38.9) > PLAY (35.4) (cells out of 80). COMMON beats every other condition with $\delta \geq 0.63$ and $p_{\text{adj}} < 0.01$. PLAY is significantly worse than every other condition (all $\delta < -0.5$, $p_{\text{adj}} < 0.03$). STR, SKILL, and RAND are not separately distinguishable on coverage ($p_{\text{adj}} > 0.38$ for all such pairs) although STR and SKILL do beat RAND on QD-score because they retain higher-fitness individuals in equivalent projected cells.

C. Cross-domain summary

The relative ordering of descriptors is not preserved across domains (Fig. 8). PLAY is 4th-best in Sokoban but worst in the platformer. STR is 3rd-best in Sokoban but tied for 2nd-best in the platformer (statistically indistinguishable from RAND on coverage). SKILL is 2nd-best in Sokoban but only 3rd in the platformer. Only COMMON (which trivially dominates by construction) and RAND (which is uniformly the no-MAP-Elites baseline) keep their rank. The intuition that “structural” descriptors are universally cheap and informative does not survive a controlled comparison.

TABLE I
Per-condition summary statistics. mean $\pm$ SD across 20 seeds; bootstrap 95% CIs for the mean in parentheses.

domain	cond	Common coverage			Common QD-score		
		mean $\pm$ SD	95% CI	wall (s)	mean $\pm$ SD	95% CI	solver calls
Sokoban	RAND	23.95 $\pm$ 0.83	[23.60, 24.30]	11.2	15.19 $\pm$ 0.59	[14.94, 15.45]	5500
Sokoban	STR	32.40 $\pm$ 2.04	[31.55, 33.35]	81.8	22.97 $\pm$ 1.00	[22.55, 23.41]	5417
Sokoban	PLAY	31.25 $\pm$ 2.53	[30.20, 32.35]	37.3	21.54 $\pm$ 1.84	[20.76, 22.31]	5416
Sokoban	SKILL	33.75 $\pm$ 2.61	[32.65, 34.90]	49.8	23.44 $\pm$ 1.80	[22.67, 24.22]	5404
Sokoban	COMMON	37.30 $\pm$ 1.66	[36.55, 38.00]	76.5	25.76 $\pm$ 1.06	[25.29, 26.21]	5382
Platformer	RAND	39.75 $\pm$ 1.52	[39.10, 40.40]	6.2	33.98 $\pm$ 1.22	[33.46, 34.48]	5000
Platformer	STR	39.90 $\pm$ 4.56	[37.90, 41.80]	5.5	35.31 $\pm$ 4.05	[33.55, 36.99]	5000
Platformer	PLAY	35.40 $\pm$ 2.89	[34.15, 36.65]	4.8	30.96 $\pm$ 2.55	[29.87, 32.05]	5000
Platformer	SKILL	38.90 $\pm$ 3.68	[37.35, 40.50]	5.7	34.27 $\pm$ 3.13	[32.95, 35.62]	5000
Platformer	COMMON	45.45 $\pm$ 3.98	[43.70, 47.15]	4.3	40.68 $\pm$ 3.57	[39.12, 42.19]	5000

Top elites by fitness — sokoban

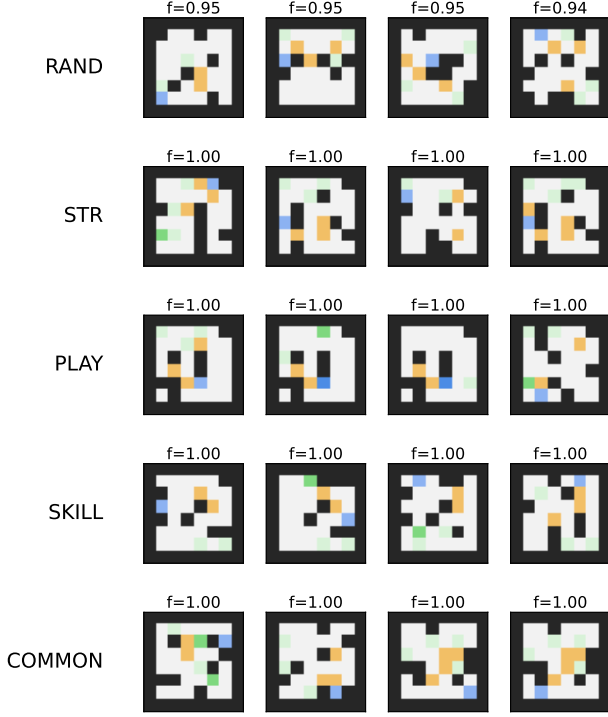


Fig. 4. Top-fitness Sokoban elites by condition (rows). Walls black, goals green, boxes orange, player blue. RAND’s best fitness saturates near 0.95; every MAP-Elites condition reaches 1.00 but on visibly different board styles. STR’s top elites have visibly denser walls than PLAY’s.

VII. Mechanism: Implicit Hardness Selection

A. Sokoban: structural pressure inflates the failure rate

The most striking finding is the wall-clock disparity in Sokoban. STR’s purely structural descriptor (boxes, wall density) never asks the solver to do anything; one might expect it to be the cheapest condition. Instead it is the slowest, at 81.8 ± 11.4 s versus 37.3 ± 11.6 s for

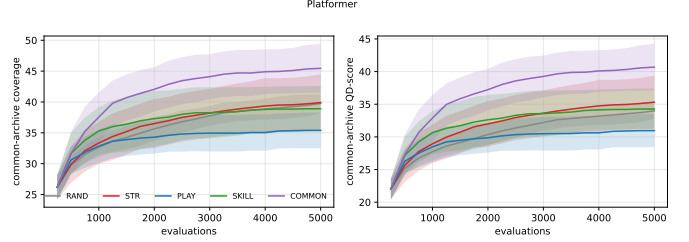


Fig. 5. Platformer: common-archive coverage (left) and QD-score (right) vs evaluations. Shaded bands are ± 1 SD across 20 seeds.

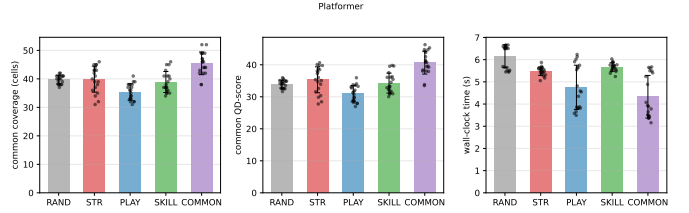


Fig. 6. Platformer final metrics by condition. Wall-clock spread is much smaller than Sokoban’s: all conditions fit within 4–6 s.

PLAY ($p_{\text{adj}} < 10^{-6}$, Cliff’s $\delta = -0.99$).

Figure 9 shows the mechanism. The grey bars give the median $\log_{10}(\text{A}^*$ nodes expanded) over all evaluated individuals (including timeouts); the coloured bars are over retained cell-winner elites. STR’s evaluated distribution sits a full decade above the others (median $\log_{10}(\text{nodes}) \approx 2.9$ vs ≈ 2.4 for PLAY), and its retained distribution remains 0.12 decades higher than PLAY’s (2.71 vs 2.59; pairwise $p_{\text{adj}} < 10^{-3}$, $\delta = -0.74$). The picture for the other three conditions is symmetric and small.

The reason is not that wall density is directly correlated with A* nodes within solvable boards (it is in fact weakly negatively correlated, Spearman $\rho = -0.07$). The reason is that selection on density preserves dense-wall elites and mutation from a dense-wall elite is disproportionately likely to produce an infeasible board — one that the solver attempts and times out on, consuming the full

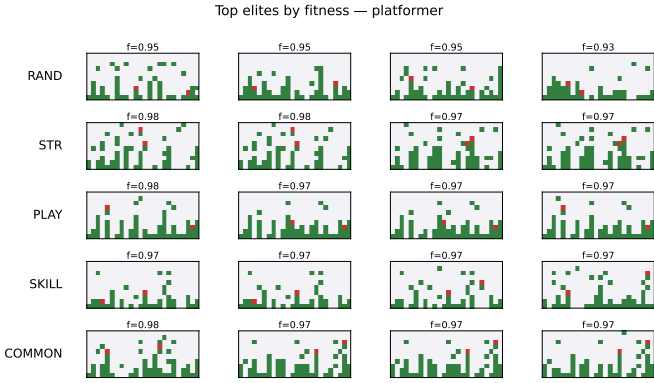


Fig. 7. Top-fitness platformer elites by condition. Green ground, red spikes, light grey sky. COMMON’s best elites have a wider variety of terrain shapes than the others.

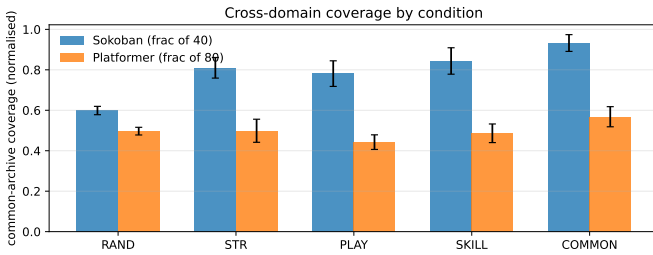


Fig. 8. Cross-domain normalised common-archive coverage. Bars show the fraction of the common archive filled, mean $\pm$ SE over 20 seeds.

8000-node budget. Table II gives the numbers: STR’s mutation pipeline produces 20.0% unsolvable-but-attempted offspring versus 12.9% for PLAY. The extra failure work, at 8000 nodes per attempt, is most of the wall-clock gap.

B. Platformer: the mechanism does not fire

The platformer’s A* over standing positions almost always terminates in well under 100 node expansions because the horizontal-distance-to-goal heuristic is tight and the underlying graph is sparse. The unsolvability rate is uniform across conditions (5–6%). Consequently there is no condition-specific failure-work inflation: Figure 10 shows the per-condition $\log_{10}(\text{nodes})$ distributions overlapping within ± 0.02 , and no pairwise difference in retained-elite hardness survives Holm correction ($p_{\text{adj}} \geq 0.41$ for all).

Critically, this is exactly what the framework predicts. The mechanism in §III states that implicit hardness selection arises when descriptor coordinates correlate with solver hardness; if the solver problem itself does not vary much, the descriptor cannot induce hardness variation either. The platformer is a case where the implicit pressure is benign — it reshapes what content gets generated (PLAY is significantly worse than COMMON in archive coverage, $p_{\text{adj}} < 10^{-5}$) without inflating the cost of generating it.

TABLE II

Per-condition mutation-outcome statistics (Sokoban, 20 seeds, 5500 evals each). %unsolv counts well-formed boards on which the solver ran and failed (timeout / infeasibility proof). $\log_{10} N_{\text{all}}$ is the mean log-node count across all solver attempts; the gap between this and $\log_{10} N_{\text{solv}}$ is the failure-driven overhead.

cond	%well-formed	%solvable	%unsolv	$\log_{10} N_{\text{all}}$
RAND	100.0%	22.2%	5.7%	2.22
STR	98.5%	55.4%	20.0%	2.90
PLAY	98.5%	63.1%	12.9%	2.45
SKILL	98.3%	64.4%	12.6%	2.42
COMMON	97.8%	62.3%	14.6%	2.54

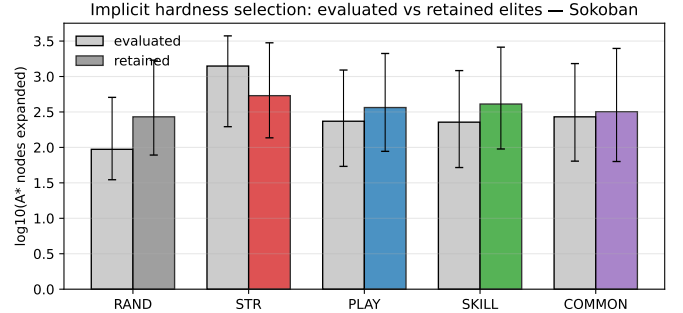


Fig. 9. Sokoban: implicit hardness selection. For each condition, the grey bar is the median $\log_{10}(A^* \text{ nodes})$ over all solvable evaluated individuals; the coloured bar is over the retained cell-winner elites. STR’s bars sit conspicuously higher than the rest. Combined with Table II, this shows the implicit selection pressure: STR both attempts more failed solves and retains harder solvable boards.

VIII. Discussion

A. What descriptor should I use?

We offer a short decision guide synthesised from the empirical results:

- 1) If you have a downstream metric you want diverse coverage on, optimise it directly. The COMMON condition (descriptor = common projection) dominates in both domains, because no implicit pressure works against you. The cost is that the resulting archive’s axes are whatever you decided to be diverse along — you get no free additional structure.
- 2) If the descriptor coordinates push selection toward regions of high infeasibility, expect wall-clock to inflate. “Structural” descriptors are not automatically cheap; in Sokoban they are $2.2\times$ slower than game-play descriptors because mutation from a dense-wall elite is $1.6\times$ more likely to produce a board the solver cannot finish, and every such attempt consumes the full node budget. The relevant correlation is not descriptor-coordinate-vs-hardness in solvable boards but descriptor-coordinate-vs-infeasibility-rate.
- 3) Gameplay descriptors are not free either. They are cheap to add (the solver’s plan is already paid for), but they may collapse into narrow corners of the common projection if their axes are nearly

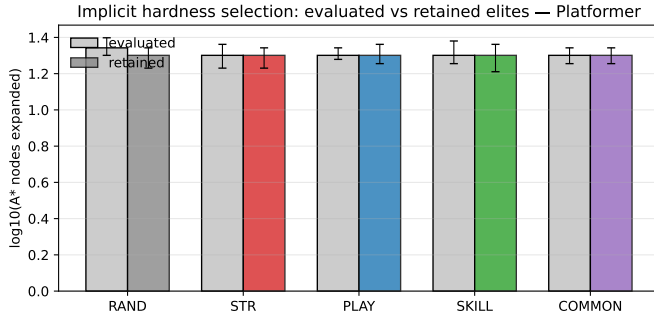


Fig. 10. Platformer: same plot as Fig. 9. No condition’s retained distribution is significantly different from any other’s.

degenerate (e.g. PLAY’s underperformance in the platformer, where plan-length spans a narrow 40–60 range so its archive column is mostly empty cells that can’t accept any individual).

- 4) Solver-effort descriptors are competitive but domain-sensitive. In Sokoban SKILL is the highest-coverage condition among the indirect descriptors; in the platformer its coverage is statistically indistinguishable from STR’s.
- 5) Don’t trust folklore. The ordering of descriptor families is empirical, not theoretical. Run the protocol of §V on a small pilot before committing to a descriptor for a larger study.

B. Why descriptor learning is not a free fix

The growing literature on learned descriptors [13]–[15] side-steps the problem we describe by making the descriptor a function of the data rather than of designer intuition. Our findings are compatible with that programme but do not subsume it: a learned descriptor that happens to correlate with solver hardness will exhibit the same implicit pressure as a hand-designed one. The learning step does not eliminate the pressure; it shifts the choice of pressure into the loss function of the descriptor learner.

C. Limitations

Two domains, with one solver in each, do not exhaust the space of PCG via QD. The descriptors we compare are intentionally simple 2D axes; “richer” descriptors (e.g. Centroidal Voronoi Tessellations [11], or learned embeddings [13]) may exhibit additional structure we do not characterise here. We also assume a uniform-random parent selection policy as in the original MAP-Elites; emitter-based variants such as CMA-ME [9] may interact with the descriptor in ways our protocol does not measure.

IX. Conclusion

The behavioral descriptor in MAP-Elites is not a passive lens onto the design space; it is a hidden selection pressure that, in domains where the descriptor pushes mutations toward infeasibility-rich regions, can silently double the wall-clock cost of a search the practitioner believed to

be cheap. The phenomenon replicates across two distinct PCG domains, with the rank-ordering of descriptor families not preserved across them. We have given a controlled experimental protocol for measuring this effect, a mechanism analysis showing it is driven by infeasibility-rate inflation rather than per-board complexity, and a practical decision guide for QD-PCG practitioners. The full code, raw archives, and per-evaluation logs are released as a replication package.

References

- [1] J. Lehman and K. O. Stanley, “Abandoning objectives: Evolution through the search for novelty alone,” *Evolutionary Computation*, vol. 19, no. 2, pp. 189–223, 2011.
- [2] J.-B. Mouret and J. Clune, “Illuminating search spaces by mapping elites,” *arXiv preprint arXiv:1504.04909*, 2015.
- [3] A. Cully and Y. Demiris, “Quality and diversity optimization: A unifying modular framework,” *IEEE Transactions on Evolutionary Computation*, vol. 22, no. 2, pp. 245–259, 2017.
- [4] J. K. Pugh, L. B. Soros, and K. O. Stanley, “Quality diversity: A new frontier for evolutionary computation,” *Frontiers in Robotics and AI*, vol. 3, p. 40, 2016.
- [5] D. Gravina, A. Khalifa, A. Liapis, J. Togelius, and G. N. Yannakakis, “Procedural content generation through quality diversity,” in *IEEE Conference on Games (CoG)*, 2019.
- [6] A. Khalifa, S. Lee, A. Nealen, and J. Togelius, “Talakat: Bullet hell generation through constrained map-elites,” in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, 2018.
- [7] M. C. Fontaine, R. Liu, J. Togelius, A. K. Hoover, and S. Nikolaidis, “Illuminating mario scenes in the latent space of a generative adversarial network,” in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2021.
- [8] S. Earle, J. Snider, M. C. Fontaine, S. Nikolaidis, and J. Togelius, “Illuminating diverse neural cellular automata for level generation,” in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, 2022.
- [9] M. C. Fontaine, J. Togelius, S. Nikolaidis, and A. K. Hoover, “Covariance matrix adaptation for the rapid illumination of behavior space,” in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, 2020, pp. 94–102.
- [10] M. C. Fontaine and S. Nikolaidis, “Covariance matrix adaptation map-annealing,” in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, 2023.
- [11] V. Vassiliades, K. Chatzilygeroudis, and J.-B. Mouret, “Using centroidal voronoi tessellations to scale up the multidimensional archive of phenotypic elites algorithm,” in *IEEE Transactions on Evolutionary Computation*, 2018.
- [12] M. C. Fontaine, Y. L. Hsu, Y. Zhang, B. Tjanaka, and S. Nikolaidis, “On the importance of environments in human-robot coordination,” in *Robotics: Science and Systems (RSS)*, 2021.
- [13] A. Cully, “Autonomous skill discovery with quality-diversity and unsupervised descriptors,” in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, 2019.
- [14] L. Grillotti and A. Cully, “Unsupervised behavior discovery with quality-diversity optimization,” *IEEE Transactions on Evolutionary Computation*, 2022.
- [15] G. Paolo, A. Laflaquière, A. Coninx, and S. Doncieux, “Unsupervised learning and exploration of reachable outcome space,” *ICRA*, 2020.
- [16] A. Summerville, S. Snodgrass, M. Guzdial, C. Holmgård, A. K. Hoover, A. Isaksen, A. Nealen, and J. Togelius, “Procedural content generation via machine learning (PCGML),” in *IEEE Transactions on Games*, 2018.
- [17] A. Khalifa, P. Bontrager, S. Earle, and J. Togelius, “PCGRL: Procedural content generation via reinforcement learning,” in *AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, 2020.
- [18] J. Culberson, “Sokoban is PSPACE-complete,” *Proceedings in Informatics 4 (Fun With Algorithms)*, 1999.

- [19] J. Taylor and I. Parberry, "Procedural generation of Sokoban levels," in *Proceedings of the International North American Conference on Intelligent Games and Simulation*, 2011.
- [20] F. Murofushi, A. M. Toda, and J. P. Borges, "Procedural generation of Sokoban levels using cellular automata," in *Brazilian Symposium on Games and Digital Entertainment (SBGames)*, 2018.
- [21] J. Kowalski, R. Miernik, K. Pelech, and M. Szykuła, "Sokoban-style level generation with evolutionary algorithms: bridging the gap between humans and machines," *arXiv preprint*, 2023.
- [22] A. Junghanns and J. Schaeffer, "Sokoban: Enhancing general single-agent search methods using domain knowledge," *Artificial Intelligence*, vol. 129, no. 1–2, pp. 219–251, 2001.
- [23] N. Shaker, G. N. Yannakakis, and J. Togelius, "Towards automatic personalized content generation for platform games," in *AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, 2010.
- [24] V. Volz, J. Schrum, J. Liu, S. M. Lucas, A. Smith, and S. Risi, "Evolving mario levels in the latent space of a deep convolutional generative adversarial network," in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, 2018.
- [25] S. Snodgrass and S. Ontañón, "Learning to generate video game maps using markov models," in *IEEE Transactions on Computational Intelligence and AI in Games*, 2017.
- [26] J. Lehman et al., "The surprising creativity of digital evolution: A collection of anecdotes from the evolutionary computation and artificial life research communities," *Artificial Life*, vol. 26, no. 2, pp. 274–306, 2020.
- [27] A. Liapis, G. N. Yannakakis, and J. Togelius, "Constrained novelty search: A study on game content generation," in *Evolutionary Computation*, 2015.
- [28] —, "Designer modeling for personalized game content creation tools," in *AAAI Workshop on Artificial Intelligence and Game Aesthetics*, 2014.
- [29] C. Holmgård, A. Liapis, J. Togelius, and G. N. Yannakakis, "Evolving personas for player decision modeling," in *IEEE Conference on Computational Intelligence and Games (CIG)*, 2014.
- [30] K. Chatzilygeroudis, A. Cully, V. Vassiliades, and J.-B. Mouret, "Quality-diversity optimization: a novel branch of stochastic optimization," *Black Box Optimization, Machine Learning, and No-Free Lunch Theorems*, 2021.
- [31] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, pp. 65–70, 1979.
- [32] J. Romano, J. D. Kromrey, J. Coraggio, J. Skowronek, and L. Devine, "Exploring methods for evaluating group differences on the NSSE and other surveys," *Annual Meeting of the Florida Association of Institutional Research*, 2006.
- [33] A. Vargha and H. D. Delaney, "A critique and improvement of the CL common language effect size statistics of McGraw and Wong," vol. 25, no. 2, 2000, pp. 101–132.